

String Matching Algorithms

SMITA SANKHE

Assistant Professor

Department of Computer Engineering

smitasankhe@somaiya.edu

String Matching Algorithms

- Naïve or Brute Force Search
- Knuth-Morris-Pratt Algorithm
- Longest Common Subsequence

Introduction

- String matching or searching algorithms try to find places where one or several strings (also called patterns) are found within a larger string (searched text):

- **Text:**

... try to find places where one or several strings (also...

- Pattern:** ace

... try to find places where one or several strings (also...

APPLICATION

- Finding all occurrences of a pattern in a text is a problem that arises frequently in text-editing programs.
- To search for particular patterns in DNA sequences.

- Input description: A text string t of length n . A pattern string p of length m .
- Problem description: Find the first (or all) instances of the pattern in the text.

"You will always have my love,
my love, for the love I love is
lovely as love itself."

love ?

"You will always have my love ,
my love , for the love I love
is love ly as love itself."

INPUT

OUTPUT

Naïve string matching Algorithm



SOMAIYA
VIDYAVIHAR UNIVERSITY

K J Somaiya College of Engineering



Naïve string matching Algorithm

- **String matching problem:** Given a text string t and a pattern string p , find all occurrences of p in t
- • A naive algorithm for this problem simply considers all possible starting positions i of a matching string within t , and compares p to the substring of t beginning at each such position I

Naive algorithm for Pattern Searching (Brute-Force Algorithm)

- The brute force algorithm consists in checking, at all positions in the text between 0 and $n-m$, whether an occurrence of the pattern starts there or not. Then, after each attempt, it shifts the pattern by exactly one position to the right.

- Example** (1)

Input: txt[] = "AABAACAADAABAABA"

pat[] = "AABA"

Text : A A B A A C A A D A A B A A B A

Pattern : A A B A

Output: Pattern found at index 0

Pattern found at index 9

Pattern found at index 12

A A B A A A B A
A A B A A C A A D A A B A A B A
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
A A B A

Pattern Found at 0, 9 and 12

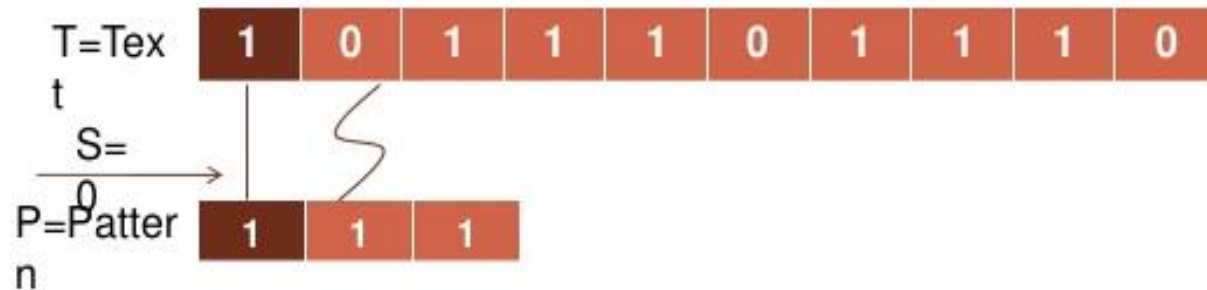
Example (2)

- SUPPOSE,

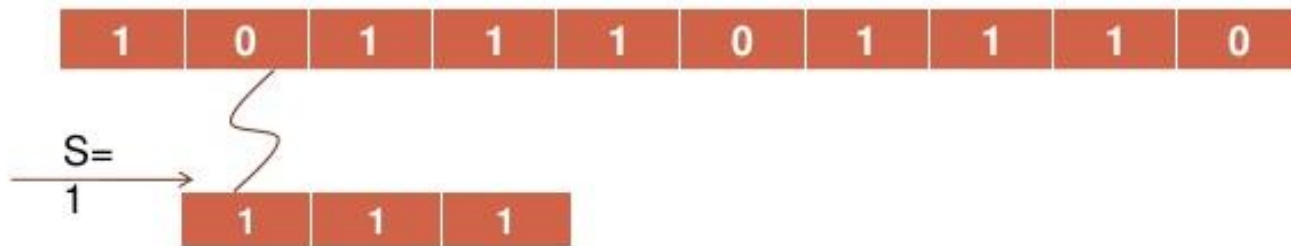
T=1011101110

P=111

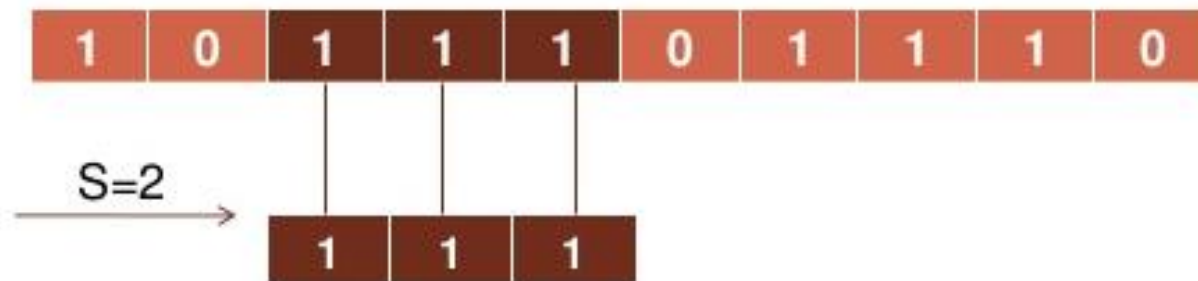
FIND ALL VALID SHIFT.....



Example (2)

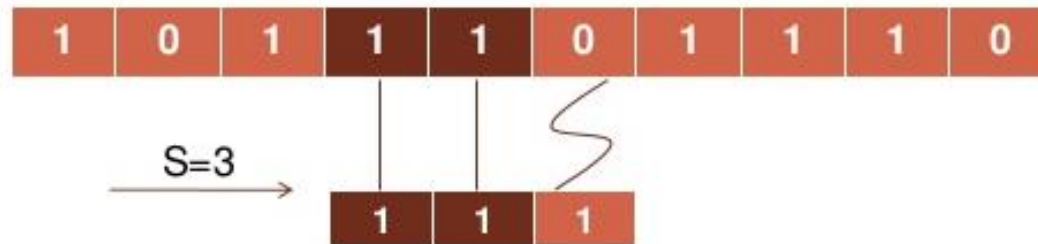


Example (2)

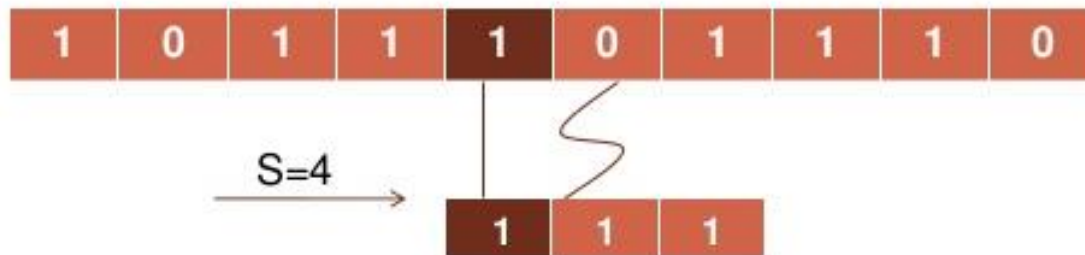


So, $S=2$ is a valid shift...

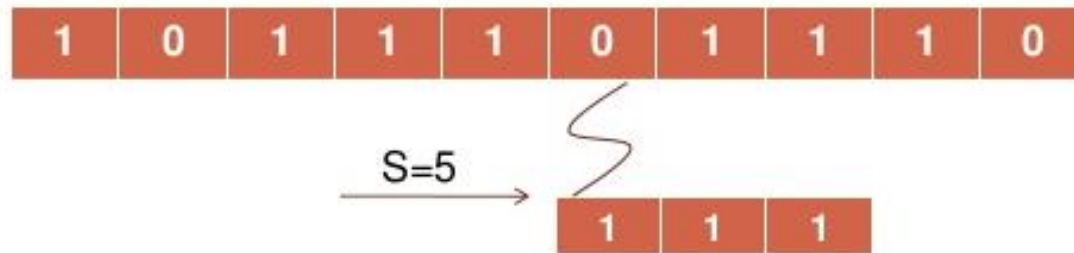
Example (2)



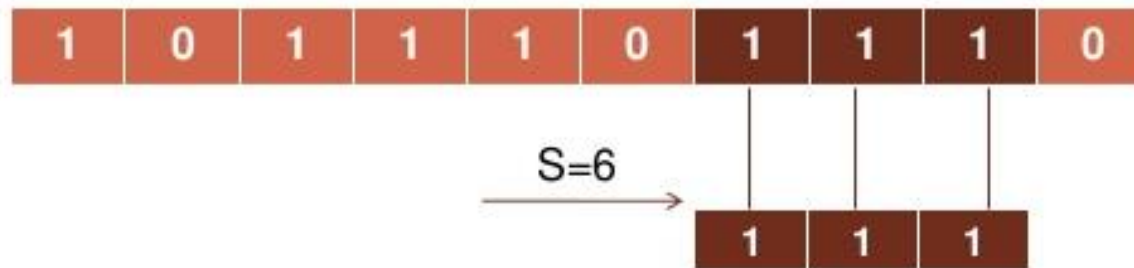
Example (2)



Example (2)

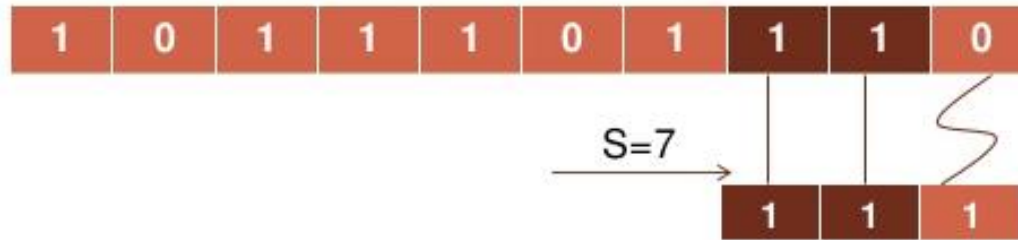


Example (2)



So, $S=6$ is a valid shift...

Example (2)



Main Features

- no preprocessing phase;
- constant extra space needed;
- always shifts the window by exactly 1 position to the right;
- comparisons can be done in any order;
- searching phase in $O(mn)$ time complexity;
- $2n$ expected text characters comparisons.

Algorithm

NAIVE-STRING-MATCHER (T, P)

1. $n \leftarrow \text{length}[T]$
2. $m \leftarrow \text{length}[P]$
3. for $s \leftarrow 0$ to $n - m$
4. do if $P[1 \dots m] = T[s + 1 \dots s + m]$
5. then print "Pattern occurs with shift" s

The naive algorithm finds all valid shifts using a loop that checks the condition $P[1 \dots m] = T[s+1 \dots s+m]$ for each of the $n - m + 1$ possible values of s .

(P =pattern , T =text/string , s =shift)

- **What is the best case?**

- The best case occurs when the first character of the pattern is not present in text at all.

txt[] = "AABCCAADDEE";

pat[] = "FAA";

- The number of comparisons in best case is **$O(n)$** .

- **What is the worst case ?**

The worst case of Naive Pattern Searching occurs in following scenarios.

1) When all characters of the text and pattern are same.

`txt[] = "AAAAAAAAAAAAAAAAAAAAA";`

`pat[] = "AAAAA";`

2) Worst case also occurs when only the last character is different.

`txt[] = "AAAAAAAAAAAAAAAAAAAB";`

`pat[] = "AAAAB";`

- The number of comparisons in the worst case is

$O(m*(n-m+1))$. Although strings which have repeated characters are not likely to appear in English text, they may well occur in other applications (for example, in binary texts).

The Knuth-Morris-Pratt (KMP) Algorithm

The Knuth-Morris-Pratt (KMP) Algorithm

- Knuth-Morris and Pratt introduce a linear time algorithm for the string matching problem.
- A matching time of $O(n)$ is achieved by avoiding comparison with an element of 'S' that have previously been involved in comparison with some element of the pattern 'p' to be matched. i.e., backtracking on the string 'S' never occurs

Components of KMP Algorithm:

- 1. The Prefix Function (Π):** The Prefix Function, Π for a pattern encapsulates knowledge about how the pattern matches against the shift of itself. This information can be used to avoid a useless shift of the pattern 'p.' In other words, this enables avoiding backtracking of the string 'S.'
- 2. The KMP Matcher:** With string 'S,' pattern 'p' and prefix function ' Π ' as inputs, find the occurrence of 'p' in 'S' and returns the number of shifts of 'p' after which occurrences are found.

PREFIX- FUNCTION (P)

- Following pseudo code compute the prefix function, Π :

COMPUTE- PREFIX- FUNCTION (P)

1. $m \leftarrow \text{length}[P]$ // 'p' pattern to be matched
2. $\Pi[1] \leftarrow 0$
3. $k \leftarrow 0$
4. for $q \leftarrow 2$ to m
5. do while $k > 0$ and $P[k + 1] \neq P[q]$
6. do $k \leftarrow \Pi[k]$
7. If $P[k + 1] = P[q]$
8. then $k \leftarrow k + 1$
9. $\Pi[q] \leftarrow k$
10. Return Π

- **Running Time Analysis:**

- In the above pseudo code for calculating the prefix function, the for loop from step 4 to step 10 runs 'm' times.
- Step1 to Step3 take constant time. Hence the running time of computing prefix function is **$O(m)$** .

- **Example:** Compute Π for the pattern 'p' below:

P:

a	b	a	b	a	c	a
---	---	---	---	---	---	---

- **Solution**
- **Initially:** $m = \text{length } [p] = 7$
- $\Pi [1] = 0$
- $k = 0$

Step 1: $q = 2, k = 0$

$$\Pi[2] = 0$$

q	1	2	3	4	5	6	7
p	a	b	a	b	a	c	a
π	0	0					

Step 2: $q = 3, k = 0$

$$\Pi[3] = 1$$

q	1	2	3	4	5	6	7
p	a	b	a	b	a	c	a
π	0	0	1				

Step3: $q = 4, k = 1$

$$\Pi[4] = 2$$

q	1	2	3	4	5	6	7
p	a	b	a	b	a	c	A
π	0	0	1	2			

Step4: $q = 5, k = 2$

$$\Pi[5] = 3$$

q	1	2	3	4	5	6	7
p	a	b	a	b	a	c	a
π	0	0	1	2	3		

Step5: $q = 6, k = 3$

$$\Pi[6] = 0$$

q	1	2	3	4	5	6	7
p	a	b	a	b	a	c	a
π	0	0	1	2	3	0	

Step6: $q = 7, k = 1$

$$\Pi[7] = 1$$

q	1	2	3	4	5	6	7
p	a	b	a	b	a	c	a
π	0	0	1	2	3	0	1

- After iteration 6 times, the prefix function computation is complete:

q	1	2	3	4	5	6	7
p	a	b	A	b	a	c	a
π	0	0	1	2	3	0	1

The KMP Matcher:

- The KMP Matcher with the pattern 'p,' the string 'S' and prefix function ' Π ' as input, finds a match of p in S. Following pseudo code compute the matching component of KMP algorithm:

KMP-MATCHER (T, P)

1. $n \leftarrow \text{length}[T]$
2. $m \leftarrow \text{length}[P]$
3. $\Pi \leftarrow \text{COMPUTE-PREFIX-FUNCTION}(P)$
4. $q \leftarrow 0$ // numbers of characters matched
5. for $i \leftarrow 1$ to n // scan S from left to right
6. do while $q > 0$ and $P[q + 1] \neq T[i]$
7. do $q \leftarrow \Pi[q]$ // next character does not match
8. If $P[q + 1] = T[i]$
9. then $q \leftarrow q + 1$ // next character matches
10. If $q = m$ // is all of p matched?
11. then print "Pattern occurs with shift" $i - m$
12. $q \leftarrow \Pi[q]$ // look for the next match

Running Time Analysis:

- The for loop beginning in step 5 runs 'n' times, i.e., as long as the length of the string 'S.' Since step 1 to step 4 take constant times, the running time is dominated by this for the loop. Thus running time of the matching function is **O (n).**
- Example: Given a string 'T' and pattern 'P' as follows:

- Example (2)
- Let us execute the KMP Algorithm to find whether 'P' occurs in 'T.'

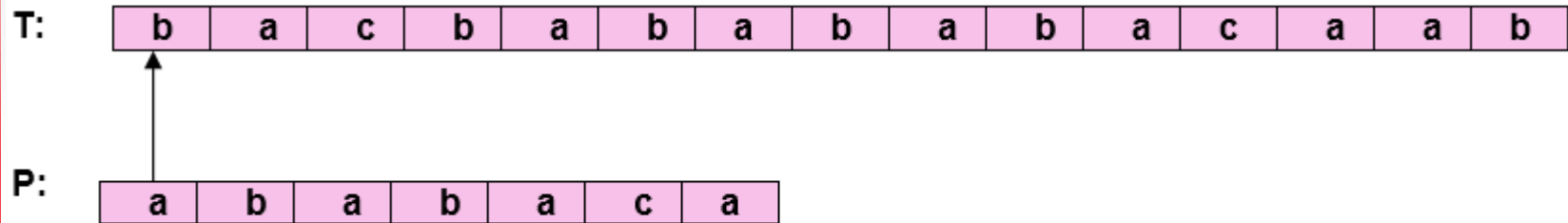
q	1	2	3	4	5	6	7
p	a	b	A	b	a	c	a
π	0	0	1	2	3	0	1

Solution:

Initially: $n = \text{size of } T = 15$
 $m = \text{size of } P = 7$

Step1: $i=1, q=0$

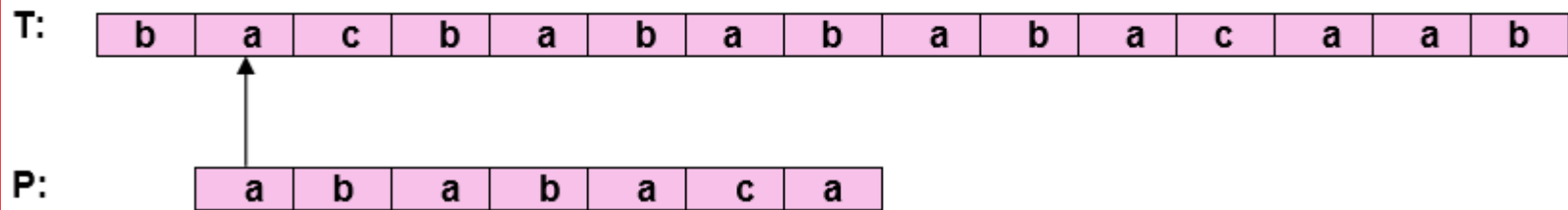
Comparing P [1] with T [1]



P [1] does not match with T [1]. 'p' will be shifted one position to the right.

Step2: $i = 2, q = 0$

Comparing P [1] with T [2]

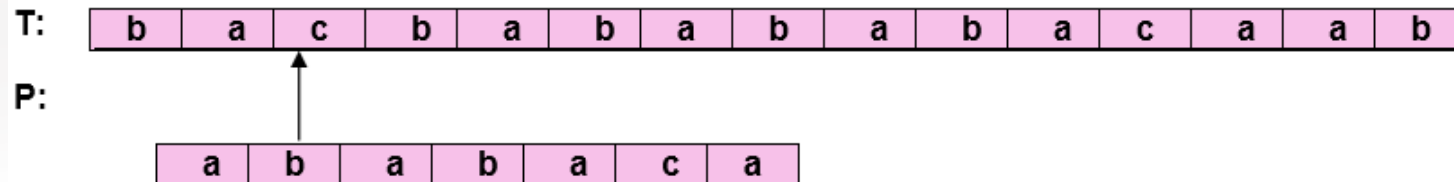


P [1] matches T [2]. Since there is a match, p is not shifted.

Step 3: $i = 3, q = 1$

Comparing P [2] with T [3]

P [2] doesn't match with T [3]

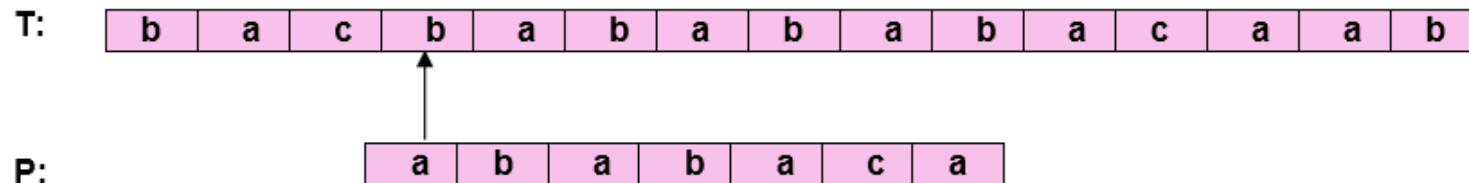


Backtracking on p, Comparing P [1] and T [3]

Step4: $i = 4, q = 0$

Comparing P [1] with T [4]

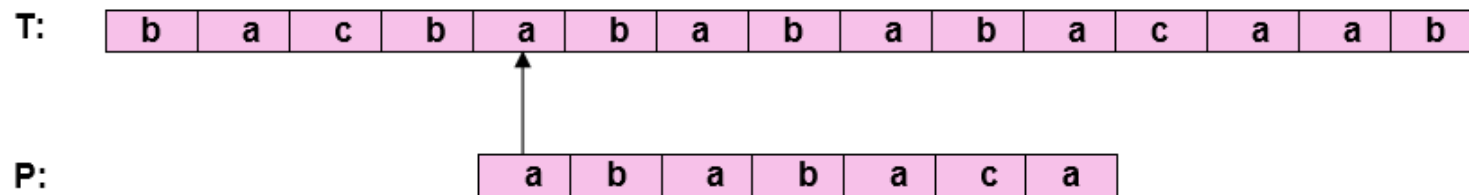
P [1] doesn't match with T [4]



Step5: $i = 5, q = 0$

Comparing P [1] with T [5]

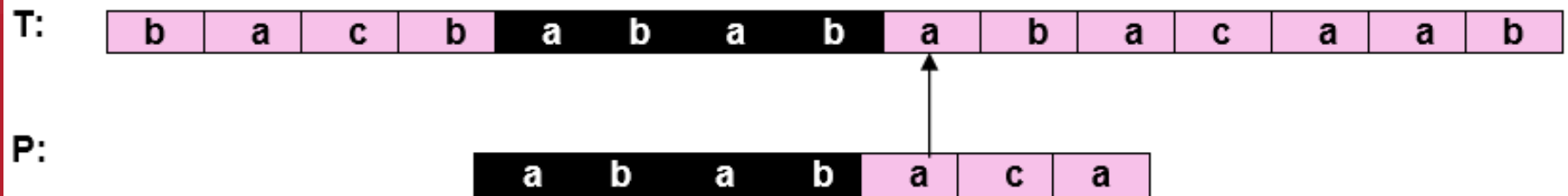
P [1] match with T [5]



Step9: $i = 9, q = 4$

Comparing P [5] with T [9]

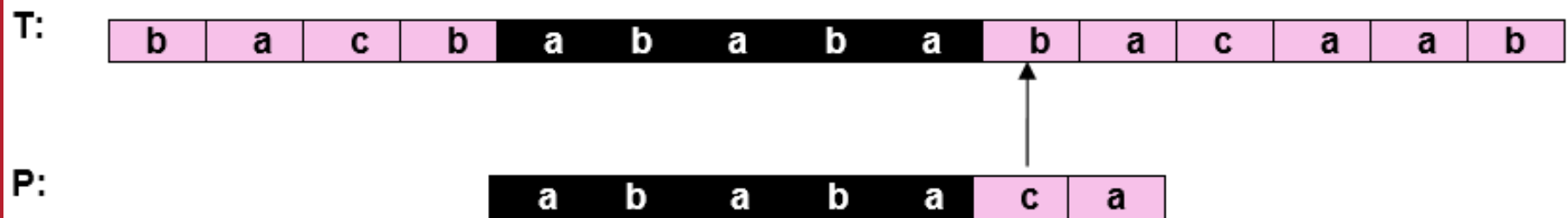
P [5] matches with T [9]



Step10: $i = 10, q = 5$

Comparing P [6] with T [10]

P [6] doesn't match with T [10]



Backtracking on p, Comparing P [4] with T [10] because after mismatch $q = \pi [5] = 3$

Step11: $i = 11, q = 4$

Comparing P [5] with T [11]

P [5] match with T [11]

T:



P:

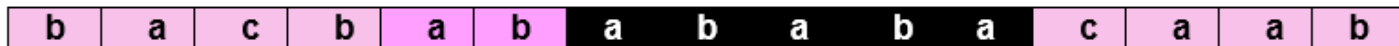


Step12: $i = 12, q = 5$

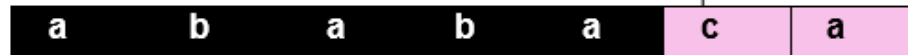
Comparing P [6] with T [12]

P [6] matches with T [12]

T:



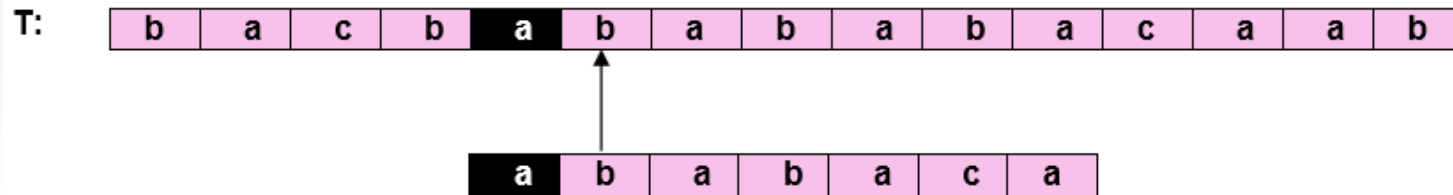
P:



Step6: $i = 6, q = 1$

Comparing P [2] with T [6]

P [2] matches with T [6]

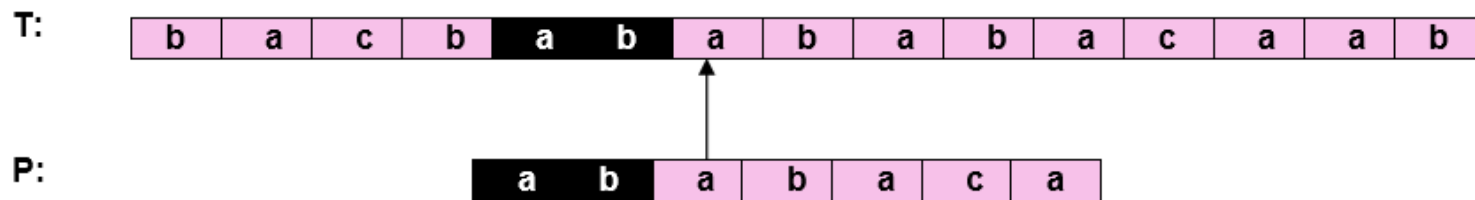


P:

Step7: $i = 7, q = 2$

Comparing P [3] with T [7]

P [3] matches with T [7]

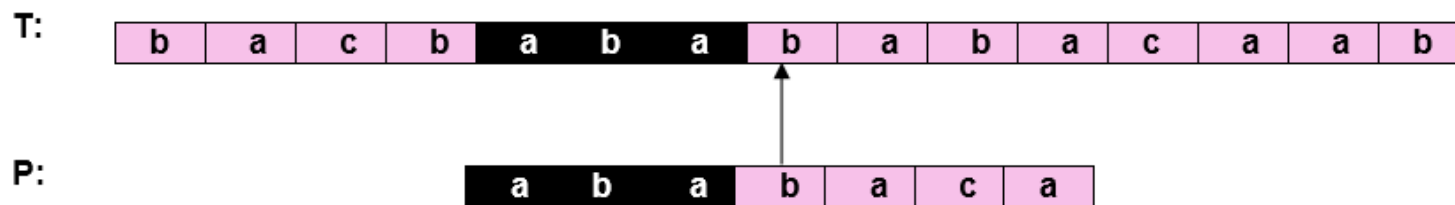


P:

Step8: $i = 8, q = 3$

Comparing P [4] with T [8]

P [4] matches with T [8]

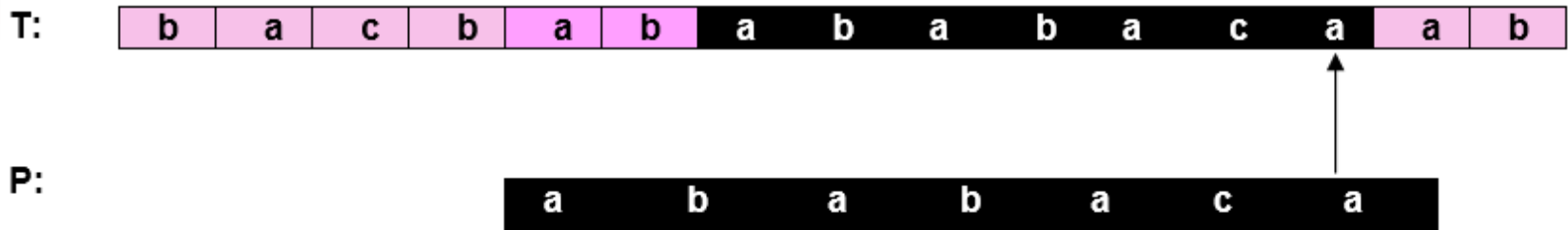


P:

Step13: $i = 3, q = 6$

Comparing P [7] with T [13]

P [7] matches with T [13]



Pattern 'P' has been found to complexity occur in a string 'T.' The total number of shifts that took place for the match to be found is $i - m = 13 - 7 = 6$ shifts.

Longest Common Subsequence Algorithm



SOMAIYA
VIDYAVIHAR UNIVERSITY

K J Somaiya College of Engineering



Longest Common Subsequence Algorithm

- **PROBLEM DEFINITION:** A subsequence of a string S , is a set of characters that appear in left –to right order, but not necessarily consecutively.
- Example
- ACTTGCG
- • ACT , ATTC , T , ACTTGC are all subsequences.
- • TTA is not a subsequence
- • There are 2^n subsequences of string S of length n .

What is LCS?

- A common subsequence of two strings is a subsequence that appears in both strings. A longest common subsequence is a common subsequence of maximal length.

Example

S1 = AAACCGTGAGTTATTCTAGAA

S2 = CACCCCTAAGGTACCTTTGGTTC

- LCS is ACCTAGTACTTTG
- Enumerate all sub-sequences of S1, and check if they are sub-sequences of S2. So the complexity will be $O(2^{nm})$

OPTIMAL SUBSTRUCTURE

If last characters of both sequences match (or $X[m-1] == Y[n-1]$) then $L(X[0..m-1], Y[0..n-1]) = 1 + L(X[0..m-2], Y[0..n-2])$

- If last characters of both sequences do not match (or $X[m-1] != Y[n-1]$) then $L(X[0..m-1], Y[0..n-1]) = \text{MAX} (L(X[0..m-2], Y[0..n-1]), L(X[0..m-1], Y[0..n-2]))$

Examples:

1) Consider the input strings “AGGTAB” and “GXTXAYB”. Last characters match for the strings. So length of LCS can be written as: $L(\text{“AGGTAB”}, \text{“GXTXAYB”}) = 1 + L(\text{“AGGTA”}, \text{“GXTXAY”})$

2) Consider the input strings “ABCDGH” and “AEDFHR”. Last characters do not match for the strings. So length of LCS can be written as: $L(\text{“ABCDGH”}, \text{“AEDFHR”}) = \text{MAX} (L(\text{“ABCDG”}, \text{“AEDFHR”}), L(\text{“ABCDGH”}, \text{“AEDFH”}))$

Overlapping Subproblem

- This is a correct solution but it's very time consuming. For example, if the two strings have no matching characters, so the last line always gets executed, the the time bounds are binomial coefficients, which (if $m=n$) are close to 2^n .

lcs("AXYT", "AYZX")

/ \

lcs("AXY", "AYZX") lcs("AXYT", "AYZ")

/ \ / \

lcs("AX", "AYZX") lcs("AXY", "AYZ") lcs("AXY", "AYZ") lcs("AXYT", "AY")

- In the above partial recursion tree, lcs("AXY", "AYZ") is being solved twice.

So this problem has Overlapping Substructure property and recomputation of same subproblems can be avoided by either using Memoization or Tabulation by using "top down" approach of dynamic programming . The concept is to cache the result of a function given its parameter so that the calculation will not be repeated; it is simply retrieved, or memo-ed. Most of the time a simple array is used for the cache table, but a hash table or map could also be employed.

LCS Recursive equation

Theorem:

Let $X = \langle x_1, x_2, \dots, x_m \rangle$ and $Y = \langle y_1, y_2, \dots, y_n \rangle$ be sequences, and let $Z = \langle z_1, z_2, \dots, z_k \rangle$ be any LCS of X and Y .

1. If $x_m = y_n$, then $z_k = x_m = y_n$ and Z_{k-1} is an LCS of X_{m-1} and Y_{n-1} .
2. If $x_m \neq y_n$, then $z_k \neq x_m$ implies that Z is an LCS of X_{m-1} and Y_n .
3. If $x_m \neq y_n$, then $z_k \neq y_n$ implies that Z is an LCS of X and Y_{n-1} .

- So recursive solution for LCS IS

0 if $i = 0$ or $j = 0$

$c[i, j] = c[i - 1, j - 1] + 1$ if $i, j > 0$ and $x_i = y_j$,

$\max(c[i, j - 1], c[i - 1, j])$ if $i, j > 0$ and $x_i \neq y_j$

Algorithm

LCS – Length(X, Y)

m= length[X]

n= length[Y]

for i =1 to m

 c[i, 0] = 0

for j =0 to n

 c[0, j] = 0

for i = 1 to m

 for j = 1 to n

 if $x_i = y_j$

c[i, j] = c[i - 1, j - 1] + 1

 B[i, j] := 'D' or ↖

 else if c[i - 1, j] > c[i, j - 1]

c[i, j] = c[i - 1, j]

 B[i, j] := 'U' or ↑

 else

c[i, j] = c[i, j - 1]

 B[i, j] := 'L' or ←

 return c and B

SEQUENCE RETRIEVAL

Algorithm: Print-LCS (B, X, i, j)

if $i = 0$ and $j = 0$

return

if $B[i, j] = 'D'$

 Print-LCS(B, X, $i-1, j-1$)

 Print(x_i)

else if $B[i, j] = 'U'$

 Print-LCS(B, X, $i-1, j$)

else

 Print-LCS(B, X, $i, j-1$)

Example:

- We have two strings $X = \text{ABCBDAB}$ and $Y = \text{BDCABA}$ to find the longest common subsequence. Following the algorithm LCS Length-

		j	0	1	2	3	4	5	6
		y_j		B	D	C	A	B	A
0	x_i		0	0	0	0	0	0	0
1	A		0	↑	↑	↑	↖	←	↖
2	B		0	↖	←	←	↑	↖	←
3	C		0	↑	↑	↖	←	↑	↑
4	B		0	↖	↑	↑	↑	↖	←
5	D		0	↑	↖	↑	↑	↑	↑
6	A		0	↑	↑	↑	↖	↑	↖
7	B		0	↖	↑	↑	↑	↖	↑

Time complexity analysis

- Each call to subproblem takes constant time. We call it once from the main routine, and at most twice every time we fill in an entry of array L. There are $(m)(n)$ entries, so the total number of calls is at most $2(m)(n)$ and the time is $O(mn)$.
- As usual, this is a worst case analysis. The time might sometimes be better, if not all array entries get filled out. For instance if the two strings match exactly, we'll only fill in diagonal entries and the algorithm will be fast.